DESARROLLO DE APLICACIONES DISTRIBUIDAS · UNIDAD II

S8 — Mensajería asíncrona entre servicios

Comunicación basada en eventos con **Apache Kafka**: **citas-ms** publica `cita.creada`, el nuevo **veterinaria-pago-ms** lo consume, registra el pago y publica `pago.validado`, y **citas-ms** marca la cita como `PAGADA`. Ningún servicio espera al otro: el registro de citas funciona aunque el servicio de pagos esté caído.

ESTUDIANTE

Joaquín Hassan Sanchez Saravia

SESIÓN

S8 · Mensajería asíncrona entre servicios

FECHA

3 de octubre de 2026

CONTENIDO

1. Introducción: síncrono vs asíncrono
2. Arquitectura y flujo de eventos
3. Infraestructura Kafka (DEV y PROD)
4. Tópicos y prueba con Python
5. veterinaria-pago-ms
6. Integración en citas-ms
7. Contratos de eventos
8. Prueba end-to-end
9. Evidencia de desacoplamiento
10. Robustez del consumidor
11. Preguntas de defensa
12. Criterios de aceptación, conclusiones y bibliografía

01

Introducción: síncrono vs asíncrono

En S6, `citas-ms` consulta a `veterinaria-mascota-ms` con **Feign**: la llamada es síncrona, el que llama espera la respuesta y ambos servicios deben estar disponibles a la vez. Eso es correcto cuando el dato se necesita en ese momento (no se puede crear una cita para una mascota que no existe), pero acopla la disponibilidad de los dos servicios; por eso fue necesario el Circuit Breaker.

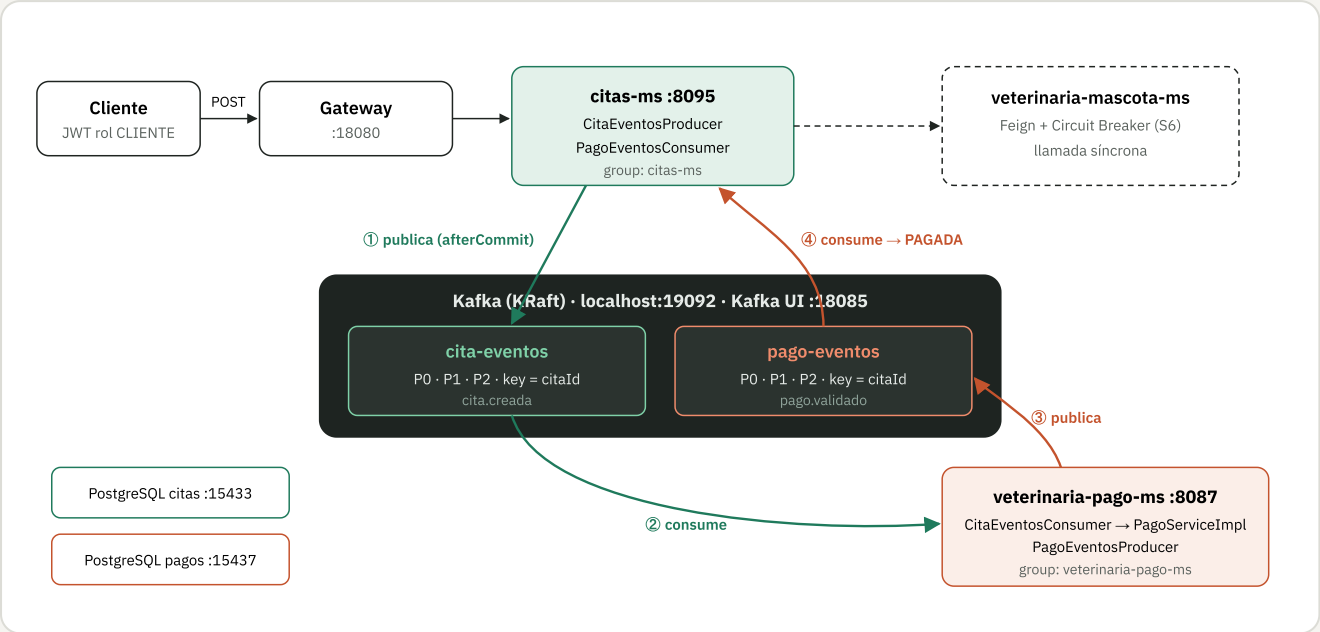
El cobro de la cita no necesita respuesta inmediata. En S8 se resuelve con **eventos**: `citas-ms` publica un hecho (*"se creó la cita 4 por S/ 95.50 con YAPE"*) en un tópico de Kafka y responde `201` al cliente sin esperar. `veterinaria-pago-ms` lo procesa cuando puede. La consistencia pasa a ser **eventual**: durante unos milisegundos (o minutos, si pagos está caído) la cita está en `PENDIENTE_PAGO`, y luego pasa a `PAGADA`.

Síncrono (Feign, S6)	Asíncrono (Kafka, S8)

Espera	El que llama se bloquea hasta la respuesta	El productor publica y continúa
Disponibilidad	Ambos servicios deben estar arriba	El consumidor puede estar caído; el mensaje espera en el broker
Consistencia	Inmediata	Eventual
Uso en el sistema	Validar la mascota al crear la cita	Registrar el pago y actualizar el estado de la cita

02

Arquitectura y flujo de eventos



Flujo: ① `citas-ms` guarda la cita en `PENDIENTE_PAGO` y, tras el commit, publica `cita.creada`; ② `veterinaria-pago-ms` la consume y guarda el pago; ③ tras su commit publica `pago.validado`; ④ `citas-ms` lo consume y cambia la cita a `PAGADA`. Cada servicio tiene su propia base de datos y su propio *consumer group*.

Componentes de Kafka usados: el **broker** (un nodo en modo KRaft, sin ZooKeeper), dos **tópicos** con 3 **particiones** cada uno, dos **consumer groups** (`veterinaria-pago-ms` y `citas-ms`) y la **key** = `citaId`, que manda todos los eventos de una misma cita a la misma partición y conserva su orden. El **offset** confirmado de cada grupo indica hasta qué mensaje se procesó.

03

Infraestructura Kafka (DEV y PROD)

Se agregó `infra/kafka/compose-dev.yml` con Kafka 3.9 (imagen `apache/kafka`) en modo KRaft y Kafka UI. El broker expone dos *listeners*: **EXTERNAL** (`localhost:19092`) para los microservicios que corren en el host, e **INTERNAL** (`kafka:9092`) para los contenedores de la misma red (Kafka UI y el contenedor de pruebas Python). La autocreación de tópicos está desactivada: se crean explícitamente con 3 particiones.

```
services:
  kafka:
    image: apache/kafka:3.9.1
    container_name: veterinaria-kafka-dev
    restart: unless-stopped
    ports:
      - "19092:19092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: INTERNAL://:9092,EXTERNAL://:19092,CONTROLLER://:9093
```

```
      KAFKA_ADVERTISED_LISTENERS: INTERNAL://kafka:9092,EXTERNAL://localhost:19092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT,CONTROLLER:PLAINTEXT
      KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
      # Los topicos se crean de forma explicita (consola o KafkaTopicsConfig) con 3
particiones.
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"
      KAFKA_NUM_PARTITIONS: 3
    volumes:
      - kafka_dev_data:/var/lib/kafka/data
```

Entorno	Broker (host)	Kafka UI	Bootstrap de los servicios	Archivo
DEV	`localhost:19092`	`localhost:18085`	`localhost:19092`	`infra/kafka/compose-dev.yml`
PROD	`localhost:29092`	`localhost:28085`	`veterinaria-kafka:9092` (red `veterinaria-net`)	`infra/kafka/compose-prod.yml`

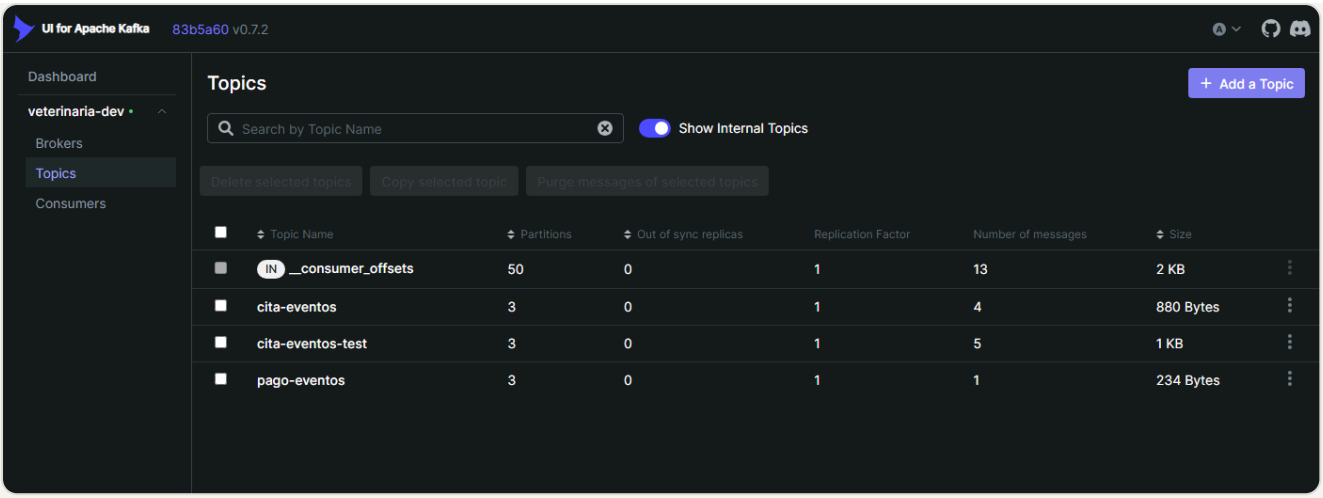
En PROD el *bootstrap* se puede sobrescribir con la variable `KAFKA_BOOTSTRAP_SERVERS` (`citas-ms-prod.yml` y `veterinaria-pago-ms-prod.yml`).

04

Tópicos y prueba con Python

Creación por consola (3.3)

```
$ docker exec veterinaria-kafka-dev /opt/kafka/bin/kafka-topics.sh --create --topic cita-
eventos --bootstrap-server kafka:9092 --partitions 3 --replication-factor 1
Created topic cita-eventos.
$ ... kafka-topics.sh --create --topic pago-eventos --partitions 3 --replication-factor 1
Created topic pago-eventos.
$ ... kafka-topics.sh --describe --topic cita-eventos
Topic: cita-eventos     TopicId: hk1MItLLS9CvCSZlFcIHzA PartitionCount: 3
ReplicationFactor: 1    Configs:
        Topic: cita-eventos     Partition: 0    Leader: 1       Replicas: 1     Isr: 1  Elr:
LastKnownElr:
        Topic: cita-eventos     Partition: 1    Leader: 1       Replicas: 1     Isr: 1  Elr:
LastKnownElr:
        Topic: cita-eventos     Partition: 2    Leader: 1       Replicas: 1     Isr: 1  Elr:
LastKnownElr:
Topic: pago-eventos     TopicId: cYo04bd6SvefW-mwfHZ4Fw PartitionCount: 3
ReplicationFactor: 1    Configs:
        Topic: pago-eventos     Partition: 0    Leader: 1       Replicas: 1     Isr: 1  Elr:
LastKnownElr:
        Topic: pago-eventos     Partition: 1    Leader: 1       Replicas: 1     Isr: 1  Elr:
LastKnownElr:
        Topic: pago-eventos     Partition: 2    Leader: 1       Replicas: 1     Isr: 1  Elr:
LastKnownElr:
```



Kafka UI (http://localhost:18085): `cita-eventos` y `pago-eventos` con 3 particiones y factor de replicación 1. `cita-eventos-test` es el tópico de la prueba Python.

Productor y consumidor Python (3.6)

El contenedor `eventos-py` (perfil `pruebas`, librería `kafka-python-ng`) publica un `cita.creada` cada 2 segundos y un consumidor valida que cada mensaje tenga los campos del contrato. Se usó un tópico aparte (`cita-eventos-test`) para que estos mensajes de prueba, con `citaId` inexistentes, no generen pagos reales en `veterinaria-pago-ms`.

```
$ docker compose --profile pruebas run --rm -e TOPICO=cita-eventos-test eventos-py python -u
productor.py
[productor] enviado citaId=9000 particion=1 offset=0 -> {'tipoEvento': 'cita.creada', 'citaId':
9000, 'idCliente': 7, 'mascotaId': 1, 'monto': 120.0, 'metodoPago': 'EFECTIVO', 'origen':
'eventos-py', 'timestamp': 1791077142107}
[productor] enviado citaId=9001 particion=0 offset=0 -> {'tipoEvento': 'cita.creada', 'citaId':
9001, 'idCliente': 7, 'mascotaId': 1, 'monto': 80.0, 'metodoPago': 'YAPE', 'origen': 'eventos-
py', 'timestamp': 1791077144137}
[productor] enviado citaId=9002 particion=1 offset=1 -> {'tipoEvento': 'cita.creada', 'citaId':
9002, 'idCliente': 7, 'mascotaId': 1, 'monto': 80.0, 'metodoPago': 'TARJETA', 'origen':
'eventos-py', 'timestamp': 1791077146141}
[productor] enviado citaId=9003 particion=2 offset=0 -> {'tipoEvento': 'cita.creada', 'citaId':
9003, 'idCliente': 7, 'mascotaId': 1, 'monto': 120.0, 'metodoPago': 'EFECTIVO', 'origen':
'eventos-py', 'timestamp': 1791077148145}
[productor] enviado citaId=9004 particion=2 offset=1 -> {'tipoEvento': 'cita.creada', 'citaId':
9004, 'idCliente': 7, 'mascotaId': 1, 'monto': 80.0, 'metodoPago': 'TARJETA', 'origen':
'eventos-py', 'timestamp': 1791077150148}
[productor] fin

$ docker compose --profile pruebas run --rm -e TOPICO=cita-eventos-test eventos-py python -u
consumidor.py
[consumidor] particion=2 offset=0 key=9003 VALIDO -> {'tipoEvento': 'cita.creada', 'citaId':
9003, 'idCliente': 7, 'mascotaId': 1, 'monto': 120.0, 'metodoPago': 'EFECTIVO', 'origen':
'eventos-py', 'timestamp': 1791077148145}
[consumidor] particion=2 offset=1 key=9004 VALIDO -> {'tipoEvento': 'cita.creada', 'citaId':
9004, 'idCliente': 7, 'mascotaId': 1, 'monto': 80.0, 'metodoPago': 'TARJETA', 'origen':
'eventos-py', 'timestamp': 1791077150148}
[consumidor] particion=0 offset=0 key=9001 VALIDO -> {'tipoEvento': 'cita.creada', 'citaId':
9001, 'idCliente': 7, 'mascotaId': 1, 'monto': 80.0, 'metodoPago': 'YAPE', 'origen': 'eventos-
py', 'timestamp': 1791077144137}
[consumidor] particion=1 offset=0 key=9000 VALIDO -> {'tipoEvento': 'cita.creada', 'citaId':
9000, 'idCliente': 7, 'mascotaId': 1, 'monto': 120.0, 'metodoPago': 'EFECTIVO', 'origen':
'eventos-py', 'timestamp': 1791077142107}
[consumidor] particion=1 offset=1 key=9002 VALIDO -> {'tipoEvento': 'cita.creada', 'citaId':
```

```
9002, 'idCliente': 7, 'mascotaId': 1, 'monto': 80.0, 'metodoPago': 'TARJETA', 'origen':
'eventos-py', 'timestamp': 1791077146141}
[consumidor] sin mensajes nuevos, fin
```

Se observa la distribución por *key*: los 5 mensajes se reparten entre las particiones 0, 1 y 2, y el consumidor los lee en orden dentro de cada partición (no entre particiones).

05

veterinaria-pago-ms

Nuevo microservicio en `services/veterinaria-pago-ms` (Spring Boot 4.0.7, Java 21, paquete `pe.edu.upeu.pago`, puerto **8087**). Dependencias: Web MVC, Validation, Lombok, Data JPA, PostgreSQL, Flyway, Config Client, Eureka Client, **Kafka**, OAuth2 Resource Server, Actuator, Prometheus y **MapStruct 1.6.3** (agregado a mano junto con `lombok-mapstruct-binding`). Base propia `veterinaria_pago_db` en el contenedor `veterinaria-postgres-pago-dev` (puerto **15437**, porque 15435 ya lo usa la base de auth).

```
veterinaria-pago-ms/src/main/java/pe/edu/upeu/pago
├── config/      KafkaTopicsConfig, SecurityConfig (permitAll en S8)
├── controller/  PagoController            GET /api/v1/pagos, GET /api/v1/pagos/{id}
├── dto/         PagoResponse
├── entity/      Pago, EstadoPago (VALIDADO)
├── event/       CitaCreadaEvento (consume), PagoValidadoEvento (publica)
├── exception/   GlobalExceptionHandler, ResourceNotFoundException
├── filter/      CorrelationIdFilter
├── kafka/       CitaEventosConsumer, PagoEventosProducer
├── mapper/      PagoMapper (MapStruct)
├── repository/  PagoRepository
└── service/     PagoService, PagoServiceImpl
```

Migración Flyway

```
-- Un pago por cita: cita_id UNIQUE hace idempotente el consumo del evento
-- cita.creada (si Kafka reentrega el mensaje no se duplica el pago).
CREATE TABLE IF NOT EXISTS pagos (
    id          BIGINT GENERATED BY DEFAULT AS IDENTITY,
    cita_id     BIGINT         NOT NULL,
    id_cliente  BIGINT,
    monto       NUMERIC(10, 2) NOT NULL,
    metodo_pago VARCHAR(20)    NOT NULL,
    estado      VARCHAR(20)    NOT NULL,
    fecha_pago  TIMESTAMP      NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT uk_pagos_cita_id UNIQUE (cita_id)
);
```

Configuración en Config Server (`veterinaria-pago-ms-dev.yml`)

```
    show-sql: false
  kafka:
    bootstrap-servers: localhost:19092
    consumer:
      group-id: veterinaria-pago-ms
      auto-offset-reset: earliest
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      # ErrorHandlingDeserializer: un mensaje mal formado se registra y se descarta
      # en lugar de dejar al consumidor en un bucle de errores.
      value-deserializer:
org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      properties:
```

```
        spring.deserializer.value.delegate.class:
org.springframework.kafka.support.serializer.JacksonJsonDeserializer
        spring.json.value.default.type: pe.edu.upeu.pago.event.CitaCreadaEvento
        spring.json.use.type.headers: false
        spring.json.trusted.packages: "*"
    producer:
      acks: all
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JacksonJsonSerializer
      properties:
        spring.json.add.type.headers: false

app:
  kafka:
    topics:
      cita-eventos: cita-eventos
      pago-eventos: pago-eventos
```

`ErrorHandlingDeserializer` envuelve al deserializador JSON: si un mensaje no es JSON válido, el error se registra y el mensaje se salta, en lugar de que el consumidor quede reintentando el mismo offset para siempre. Los nombres de tópicos se inyectan desde `app.kafka.topics`.

Consumidor

```
    @KafkaListener(topics = "${app.kafka.topics.cita-eventos}", groupId =
"${spring.kafka.consumer.group-id}")
    public void consumir(ConsumerRecord<String, CitaCreadaEvento> record) {
        CitaCreadaEvento evento = record.value();
        if (evento == null) {
            // Mensaje sin cuerpo (tombstone). Un JSON mal formado no llega aqui:
ErrorHandlingDeserializer
            // lo convierte en DeserializationException, DefaultErrorHandler lo registra y lo
salta.
            log.warn("component=cita-consumer eventType=desconocido citaId={} status=DESCARTADO
motivo=mensaje_vacio partition={} offset={}",
                    record.key(), record.partition(), record.offset());
            return;
        }
        log.info("component=cita-consumer eventType={} citaId={} status=RECIBIDO partition={}
offset={}",
                evento.getTipoEvento(), evento.getCitaId(), record.partition(),
record.offset());
        if (!CitaCreadaEvento.TIPO.equals(evento.getTipoEvento())) {
            log.info("component=cita-consumer eventType={} citaId={} status=IGNORADO
motivo=tipo_no_soportado",
                    evento.getTipoEvento(), evento.getCitaId());
            return;
        }
        pagoService.procesar(evento);
    }
```

Servicio: registra el pago y publica la respuesta

```
    /**
     * Registra el pago de la cita y publica pago.validado. La publicacion se
     * difiere hasta el commit (afterCommit en el productor) para no anunciar
     * un pago que luego se revierta. Si el evento llega repetido no se crea
```

```
     * un segundo pago: se reutiliza el existente y se vuelve a anunciar.
     */
    @Override
    @Transactional
    public void procesar(CitaCreadaEvento evento) {
        Pago pago = pagoRepository.findByCitaId(evento.getCitaId()).orElse(null);
        if (pago != null) {
            log.warn("component=pago-service eventType={} citaId={} status=DUPLICADO pagoId=
{}",
                    evento.getTipoEvento(), evento.getCitaId(), pago.getId());
        } else {
            pago = pagoRepository.save(Pago.builder()
                    .citaId(evento.getCitaId())
                    .idCliente(evento.getIdCliente())
                    .monto(evento.getMonto())
                    .metodoPago(evento.getMetodoPago())
                    .estado(EstadoPago.VALIDADO)
                    .fechaPago(LocalDateTime.now())
                    .build());
            log.info("component=pago-service eventType={} citaId={} status=PAGO_REGISTRADO
pagoId={} monto={} metodoPago={}",
                    evento.getTipoEvento(), evento.getCitaId(), pago.getId(), pago.getMonto(),
pago.getMetodoPago());
        }
        pagoEventosProducer.publicarPagoValidado(pago);
    }
```

Productor: publica solo después del commit

```
    /**
     * Se publica solo despues del commit de la transaccion de BD: si el
     * guardado falla y hace rollback, el evento nunca sale (sin eventos fantasma).
     */
    public void publicarPagoValidado(Pago pago) {
        PagoValidadoEvento evento = PagoValidadoEvento.builder()
                .tipoEvento(PagoValidadoEvento.TIPO)
                .citaId(pago.getCitaId())
                .pagoId(pago.getId())
                .monto(pago.getMonto())
                .metodoPago(pago.getMetodoPago())
                .estado(pago.getEstado().name())
                .origen(origen)
                .timestamp(System.currentTimeMillis())
                .build();

        if (TransactionSynchronizationManager.isSynchronizationActive()) {
            TransactionSynchronizationManager.registerSynchronization(new
TransactionSynchronization() {
                @Override
                public void afterCommit() {
                    enviar(evento);
                }
            });
        } else {
            enviar(evento);
        }
    }

    private void enviar(PagoValidadoEvento evento) {
        // La key es el citaId: todos los eventos de una misma cita caen en la misma particion.
```

```
        String key = String.valueOf(evento.getCitaId());
        kafkaTemplate.send(topico, key, evento).whenComplete((resultado, error) -> {
            if (error != null) {
                log.error("component=pago-producer eventType={} citaId={}
status=ERROR_PUBLICACION error={}",
                        evento.getTipoEvento(), evento.getCitaId(), error.toString());
            } else {
                log.info("component=pago-producer eventType={} citaId={} status=PUBLICADO
topic={} partition={} offset={}",
                        evento.getTipoEvento(), evento.getCitaId(), topico,
                        resultado.getRecordMetadata().partition(),
resultado.getRecordMetadata().offset());
            }
        });
    }
}
```

Mapper MapStruct y controlador

```
@Mapper(componentModel = "spring")
public interface PagoMapper {

    PagoResponse toResponse(Pago pago);

    List<PagoResponse> toResponseList(List<Pago> pagos);
}
@RequestMapping("/api/v1/pagos")
@RequiredArgsConstructor
public class PagoController {

    private final PagoService pagoService;

    @GetMapping
    public List<PagoResponse> listar() {
        return pagoService.listar();
    }

    @GetMapping("/{id}")
    public PagoResponse obtener(@PathVariable Long id) {
        return pagoService.obtener(id);
    }
}
```

`SecurityConfig` permite todas las peticiones (como indica la guía), pero el servicio ya trae la dependencia de Resource Server y el `issuer-uri`/`jwk-set-uri` en el Config Server. En el Gateway la ruta `/api/v1/pagos/**` exige rol `CLIENTE` o `ADMIN` y solo método GET.

Arranque (3.16)

```
2026-10-03 20:27:02.101 INFO  o.s.c.c.c.ConfigServerConfigDataLoader - Located environment:
name=veterinaria-pago-ms, profiles=[dev], label=null, version=null, state=null
2026-10-03 20:27:09.727 INFO  o.f.core.internal.command.DbValidate - Successfully validated 1
migration (execution time 00:00.046s)
2026-10-03 20:27:09.931 INFO  o.f.core.internal.command.DbMigrate - Migrating schema "public"
to version "1 - create pagos"
2026-10-03 20:27:10.006 INFO  o.f.core.internal.command.DbMigrate - Successfully applied 1
migration to schema "public", now at version v1 (execution time 00:00.031s)
```

```
2026-10-03 20:27:18.732 INFO  o.s.c.n.e.s.EurekaServiceRegistry - Registering application
VETERINARIA-PAGO-MS with eureka with status UP
2026-10-03 20:27:18.830 INFO  o.s.boot.tomcat.TomcatWebServer - Tomcat started on port 8087
(http) with context path '/'
2026-10-03 20:27:19.133 INFO  p.e.u.p.VeterinariaPagoApplication - Started
VeterinariaPagoApplication in 19.539 seconds (process running for 20.872)
2026-10-03 20:27:19.287 INFO  o.s.k.l.KafkaMessageListenerContainer - veterinaria-pago-ms:
partitions assigned: [cita-eventos-0, cita-eventos-1, cita-eventos-2]

$ curl localhost:8087/api/v1/pagos   (recien iniciado)
[]
```

06

Integración en citas-ms

- `pom.xml`: `spring-boot-starter-kafka`. Config Server: misma configuración de productor/consumidor con `group-id: citas-ms`.
- Migración `V4__add_pago_to_citas.sql`: columnas `monto` y `metodo_pago` (viajan en el evento).
- `CitaRequest`: `monto` opcional (tarifa de consulta S/ 80.00 por defecto) y `metodoPago` ∈ {TARJETA, YAPE, EFECTIVO} (EFECTIVO por defecto). Al crear, el estado ya no lo decide el cliente.
- `CitaService.crear()` es `@Transactional`: si la mascota se validó, la cita queda `PENDIENTE_PAGO` y se publica `cita.creada` después del commit. Si el Circuit Breaker de S6 está abierto, la cita queda `PENDIENTE_VALIDACION` y **no** se publica.
- `PagoEventosConsumer` escucha `pago-eventos` y llama a `CitaService.marcarPagada()`.

```
    /**
     * Si la mascota se valido, la cita queda PENDIENTE_PAGO y se publica
     * cita.creada (despues del commit). No se espera a veterinaria-pago-ms:
     * la respuesta 201 sale aunque ese servicio este apagado.
     */
    @Transactional
    public CitaResponse crear(Long idCliente, CitaRequest request) {
        log.info("Creando cita para idCliente={} (tomado del JWT), mascotaId={}, consultando
veterinaria-mascota-ms...",
                idCliente, request.getMascotaId());
        MascotaDto mascota = validarMascota(request.getMascotaId());
        Cita cita = citaMapper.toEntity(request);
        cita.setIdCliente(idCliente);
        cita.setMonto(request.getMonto() != null ? request.getMonto() : TARIFA_CONSULTA);
        cita.setMetodoPago(request.getMetodoPago() != null ? request.getMetodoPago() :
METODO_PAGO_POR_DEFECTO);
        aplicarDatosMascota(cita, mascota, ESTADO_PENDIENTE_PAGO);
        Cita guardada = citaRepository.save(cita);
        log.info("Cita {} creada para idCliente={} con mascotaNombre='{}' y estado='{}'",
                guardada.getId(), guardada.getIdCliente(), guardada.getMascotaNombre(),
guardada.getEstado());
        if (ESTADO_PENDIENTE_PAGO.equals(guardada.getEstado())) {
            citaEventosProducer.publicarCitaCreada(guardada);
        }
        return citaMapper.toResponse(guardada);
    }

    /**
     * Reaccion al evento pago.validado. Solo una cita PENDIENTE_PAGO pasa a
     * PAGADA; si el evento llega repetido la cita ya esta PAGADA y no cambia
     * (consumo idempotente).
     */
    @Transactional
    public void marcarPagada(Long citaId, Long pagoId) {
        Cita cita = citaRepository.findById(citaId).orElse(null);
        if (cita == null) {
            log.warn("component=cita-service eventType=pago.validado citaId={}
status=CITA_NO_ENCONTRADA", citaId);
            return;
        }
        if (!ESTADO_PENDIENTE_PAGO.equals(cita.getEstado())) {
```

```
            log.info("component=cita-service eventType=pago.validado citaId={}
status=SIN_CAMBIO estadoActual={}",
                    citaId, cita.getEstado());
            return;
        }
        cita.setEstado(ESTADO_PAGADA);
        citaRepository.save(cita);
        log.info("component=cita-service eventType=pago.validado citaId={} status=CITA_PAGADA
pagoId={} estado={}->{}",
                citaId, pagoId, ESTADO_PENDIENTE_PAGO, ESTADO_PAGADA);
    }
```

```
    /**
     * Publica cita.creada solo despues del commit: si la insercion de la cita
     * hace rollback, el evento no sale (sin pagos de citas inexistentes).
     */
    public void publicarCitaCreada(Cita cita) {
        CitaCreadaEvento evento = CitaCreadaEvento.builder()
                .tipoEvento(CitaCreadaEvento.TIPO)
                .citaId(cita.getId())
                .idCliente(cita.getIdCliente())
                .mascotaId(cita.getMascotaId())
                .monto(cita.getMonto())
                .metodoPago(cita.getMetodoPago())
                .origen(origen)
                .timestamp(System.currentTimeMillis())
                .build();

        if (TransactionSynchronizationManager.isSynchronizationActive()) {
            TransactionSynchronizationManager.registerSynchronization(new
TransactionSynchronization() {
                @Override
                public void afterCommit() {
                    enviar(evento);
                }
            });
        } else {
            enviar(evento);
        }
    }
```

07

Contratos de eventos (3.21)

Un evento registra un **hecho ya ocurrido**, por eso se nombra en pasado (`cita.creada`, `pago.validado`). El contrato es el acuerdo entre productor y consumidor sobre la forma del mensaje. Cada servicio tiene **su propia copia** de las clases (`pe.edu.upeu.citas.event.*` y `pe.edu.upeu.pago.event.*`): no existe una librería compartida, así cada servicio se despliega por separado. El JSON se serializa sin cabeceras de tipo (`spring.json.add.type.headers: false`), así el consumidor no depende del nombre de clase Java del productor.

cita.creada — tópico `cita-eventos`

Campo	Tipo	Significado	Ejemplo
`tipoEvento`	string	Tipo de evento; el consumidor ignora otros valores	`"cita.creada"`
`citaId`	long	Id de la cita en citas-ms. También es la **key** del mensaje	`4`
`idCliente`	long	Dueño de la cita (claim `idCliente` del JWT, S7)	`1`
`mascotaId`	long	Mascota atendida	`1`
`monto`	decimal	Importe a cobrar en soles	`95.50`
`metodoPago`	string	`TARJETA` | `YAPE` | `EFECTIVO`	`"YAPE"`
`origen`	string	Servicio que publica	`"citas-ms"`
`timestamp`	long (epoch ms)	Momento en que ocurrió el hecho	`1791077293201`

pago.validado — tópico `pago-eventos`

Campo	Tipo	Significado	Ejemplo
`tipoEvento`	string	Tipo de evento	`"pago.validado"`
`citaId`	long	Cita pagada. Es la **key** (misma partición que su `cita.creada`)	`4`
`pagoId`	long	Id del pago en veterinaria-pago-ms	`1`
`monto`	decimal	Importe cobrado	`95.50`
`metodoPago`	string	Método usado	`"YAPE"`
`estado`	string	Estado del pago (`EstadoPago`)	`"VALIDADO"`
`origen`	string	Servicio que publica	`"veterinaria-pago-ms"`
`timestamp`	long (epoch ms)	Momento de la validación	`1791077294124`

Mensajes reales leídos del broker

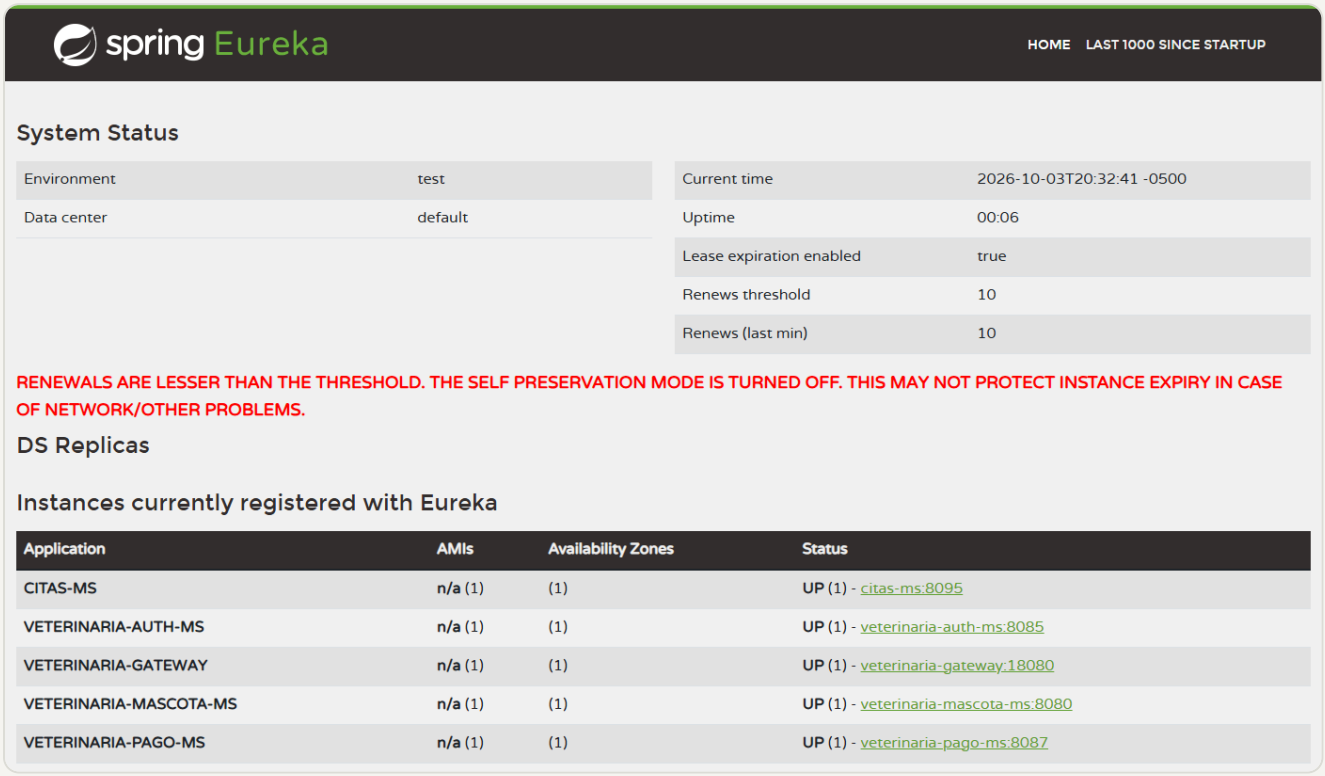
```
### cita-eventos
Partition:1     Offset:0        4
{"tipoEvento":"cita.creada","citaId":4,"idCliente":1,"mascotaId":1,"monto":95.50,"metodoPago":"
YAPE","origen":"citas-ms","timestamp":1791077293201}
Partition:1     Offset:1        6
{"tipoEvento":"cita.creada","citaId":6,"idCliente":1,"mascotaId":2,"monto":60.00,"metodoPago":"
EFECTIVO","origen":"citas-ms","timestamp":1791077330303}
Partition:1     Offset:2        4
{"tipoEvento":"cita.creada","citaId":4,"idCliente":1,"mascotaId":1,"monto":95.50,"metodoPago":"
YAPE","origen":"kafka-console","timestamp":1791077300000}
Partition:0     Offset:0        5
{"tipoEvento":"cita.creada","citaId":5,"idCliente":1,"mascotaId":1,"monto":150.00,"metodoPago":
"TARJETA","origen":"citas-ms","timestamp":1791077310244}
Partition:0     Offset:1        7
{"tipoEvento":"cita.creada","citaId":7,"idCliente":1,"mascotaId":2,"monto":45.00,"metodoPago":"
YAPE","origen":"citas-ms","timestamp":1791077330403}
Partition:0     Offset:2        99      esto-no-es-json
Partition:0     Offset:3        8
{"tipoEvento":"cita.cancelada","citaId":8,"origen":"kafka-console","timestamp":1791077300000}
### pago-eventos
Partition:0     Offset:0        5
{"tipoEvento":"pago.validado","citaId":5,"pagoId":3,"monto":150.00,"metodoPago":"TARJETA","esta
do":"VALIDADO","origen":"veterinaria-pago-ms","timestamp":1791077464982}
Partition:0     Offset:1        7
{"tipoEvento":"pago.validado","citaId":7,"pagoId":4,"monto":45.00,"metodoPago":"YAPE","estado":
"VALIDADO","origen":"veterinaria-pago-ms","timestamp":1791077464993}
Partition:1     Offset:0        4
{"tipoEvento":"pago.validado","citaId":4,"pagoId":1,"monto":95.50,"metodoPago":"YAPE","estado":
"VALIDADO","origen":"veterinaria-pago-ms","timestamp":1791077294124}
Partition:1     Offset:1        6
{"tipoEvento":"pago.validado","citaId":6,"pagoId":2,"monto":60.00,"metodoPago":"EFECTIVO","esta
do":"VALIDADO","origen":"veterinaria-pago-ms","timestamp":1791077464882}
Partition:1     Offset:2        4
{"tipoEvento":"pago.validado","citaId":4,"pagoId":1,"monto":95.50,"metodoPago":"YAPE","estado":
"VALIDADO","origen":"veterinaria-pago-ms","timestamp":1791077491013}
```

La cita 4 aparece en la partición 1 de **ambos** tópicos: la misma key produce la misma partición. Las líneas con `origen: kafka-console` y el texto `esto-no-es-json` son las pruebas de robustez de la sección 10.

08

Prueba end-to-end (3.19)

Sistema completo en ejecución: Config Server, Eureka, Gateway, `veterinaria-auth-ms`, `veterinaria-mascota-ms`, `citas-ms`, `veterinaria-pago-ms`, Kafka, Kafka UI y cinco PostgreSQL en Docker. El token es del usuario `cliente@veterinaria.com` (rol CLIENTE).



Eureka: `VETERINARIA-PAGO-MS` registrado en el puerto 8087 junto a los demás servicios.

1. Crear la cita vía Gateway

```
$ curl -X POST $GW/api/v1/citas -H "Authorization: Bearer <TOKEN CLIENTE>" \
    -d '{"mascotaId":1,"motivo":"Vacunacion antirrabica (S8)","fechaHora":"2026-10-
20T10:00:00","metodoPago":"YAPE","monto":95.50}'
HTTP/1.1 201
{"id":4,"idCliente":1,"mascotaId":1,"mascotaNombre":"Firulais","motivo":"Vacunacion antirrabica
(S8)","fechaHora":"2026-10-
20T10:00:00","estado":"PENDIENTE_PAGO","monto":95.50,"metodoPago":"YAPE"}
```

La respuesta es `201` con estado `PENDIENTE_PAGO`: `citas-ms` no esperó al servicio de pagos.

2. Recorrido del mensaje en los logs

Cada línea lleva `component`, `eventType`, `citaId` (el `ordenId` de la guía) y `status`, además de partición y offset.

```
### citas-ms (productor + consumidor)
2026-10-03 20:28:13.453 [] INFO  p.e.u.c.kafka.CitaEventosProducer - component=cita-producer
eventType=cita.creada citaId=4 status=PUBLICADO topic=cita-eventos partition=1 offset=0
2026-10-03 20:28:14.287 [] INFO  p.e.u.c.kafka.PagoEventosConsumer - component=pago-consumer
eventType=pago.validado citaId=4 status=RECIBIDO partition=1 offset=0
```

```
2026-10-03 20:28:14.339 [] INFO  p.edu.upeu.citas.service.CitaService - component=cita-service
eventType=pago.validado citaId=4 status=CITA_PAGADA pagoId=1 estado=PENDIENTE_PAGO->PAGADA

### veterinaria-pago-ms (consumidor + servicio + productor)
2026-10-03 20:28:13.478 [] INFO  p.e.u.pago.kafka.CitaEventosConsumer - component=cita-consumer
eventType=cita.creada citaId=4 status=RECIBIDO partition=1 offset=0
2026-10-03 20:28:14.122 [] INFO  p.e.u.pago.service.PagoServiceImpl - component=pago-service
eventType=cita.creada citaId=4 status=PAGO_REGISTRADO pagoId=1 monto=95.50 metodoPago=YAPE
2026-10-03 20:28:14.286 [] INFO  p.e.u.pago.kafka.PagoEventosProducer - component=pago-producer
eventType=pago.validado citaId=4 status=PUBLICADO topic=pago-eventos partition=1 offset=0
```

Todo el ciclo tardó menos de 1 segundo (20:28:13.453 → 20:28:14.339).

3. Endpoints REST y base de datos

```
$ curl http://localhost:18080/api/v1/citas/4  (Gateway, token CLIENTE)
{"id":4,"idCliente":1,"mascotaId":1,"mascotaNombre":"Firulais","motivo":"Vacunacion antirrabica
(S8)","fechaHora":"2026-10-20T10:00:00","estado":"PAGADA","monto":95.50,"metodoPago":"YAPE"}

$ curl http://localhost:18080/api/v1/pagos  (Gateway, token CLIENTE)
[{"id":1,"citaId":4,"idCliente":1,"monto":95.50,"metodoPago":"YAPE","estado":"VALIDADO","fechaP
ago":"2026-10-03T20:28:14.050735"}]

$ curl http://localhost:18080/api/v1/pagos/1
{"id":1,"citaId":4,"idCliente":1,"monto":95.50,"metodoPago":"YAPE","estado":"VALIDADO","fechaPa
go":"2026-10-03T20:28:14.050735"}

$ curl -i http://localhost:18080/api/v1/pagos/999
HTTP/1.1 404
{"error":"Not Found","message":"Pago no encontrado: 999","timestamp":"2026-10-
04T01:28:18.308128400Z","status":404}
```

```
$ psql veterinaria_citas_db
 id | id_cliente | mascota_id | estado | monto | metodo_pago
----+------------+------------+--------+-------+-------------
  4 |          1 |          1 | PAGADA | 95.50 | YAPE
(1 row)

$ psql veterinaria_pago_db
 id | cita_id | id_cliente | monto | metodo_pago |  estado  |         fecha_pago
----+---------+------------+-------+-------------+----------+----------------------------
  1 |       4 |          1 | 95.50 | YAPE        | VALIDADO | 2026-10-03 20:28:14.050735
(1 row)
```

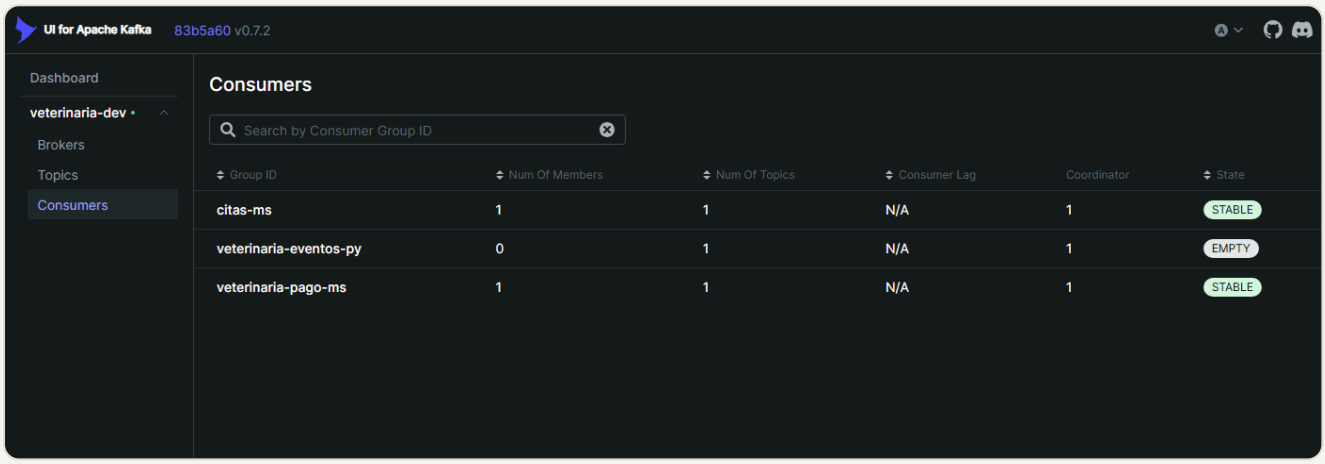
4. Consumer groups

```
$ kafka-consumer-groups.sh --describe --group veterinaria-pago-ms
GROUP               TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
CONSUMER-ID                                                     HOST            CLIENT-ID
veterinaria-pago-ms cita-eventos    1          1               1               0
consumer-veterinaria-pago-ms-1-8a4da012-64ed-4151-9e1d-0d2dabc286f2 /172.25.0.1     consumer-
veterinaria-pago-ms-1
veterinaria-pago-ms cita-eventos    0          -               0               -
consumer-veterinaria-pago-ms-1-8a4da012-64ed-4151-9e1d-0d2dabc286f2 /172.25.0.1     consumer-
veterinaria-pago-ms-1
veterinaria-pago-ms cita-eventos    2          -               0               -
consumer-veterinaria-pago-ms-1-8a4da012-64ed-4151-9e1d-0d2dabc286f2 /172.25.0.1     consumer-
```

```
veterinaria-pago-ms-1

$ kafka-consumer-groups.sh --describe --group citas-ms
GROUP           TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
CONSUMER-ID                                       HOST            CLIENT-ID
citas-ms        pago-eventos    2          -               0               -
consumer-citas-ms-1-8728620f-7d95-434c-8e69-27865a3f128b /172.25.0.1     consumer-citas-ms-1
citas-ms        pago-eventos    0          -               0               -
consumer-citas-ms-1-8728620f-7d95-434c-8e69-27865a3f128b /172.25.0.1     consumer-citas-ms-1
citas-ms        pago-eventos    1          1               1               0
consumer-citas-ms-1-8728620f-7d95-434c-8e69-27865a3f128b /172.25.0.1     consumer-citas-ms-1
```

Los dos grupos tienen **LAG 0**. Las particiones 0 y 2 muestran `-` porque todavía no habían recibido mensajes, por lo que el grupo no tenía offset confirmado allí.



Kafka UI → Consumers: los dos grupos, uno por servicio.

09

Evidencia de desacoplamiento (3.20)

1–2. Se detiene veterinaria-pago-ms y se registra una cita

```
### 1. Se detiene veterinaria-pago-ms (PID 36880)
Correcto: se termin⍰ el proceso con PID 36880.
$ curl localhost:8087/actuator/health -> 000 (000 = sin respuesta)

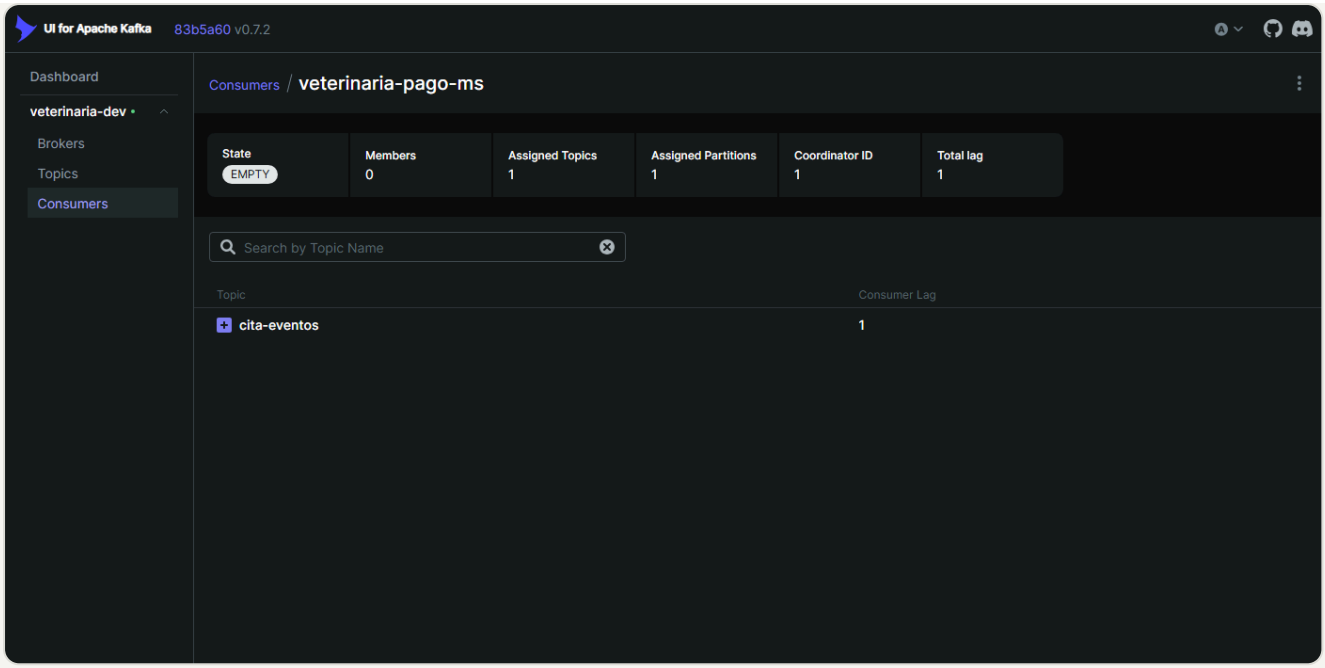
### 2. Se registra una cita con veterinaria-pago-ms apagado
HTTP/1.1 201
{"id":5,"idCliente":1,"mascotaId":1,"mascotaNombre":"Firulais","motivo":"Control post-
operatorio (pago-ms apagado)","fechaHora":"2026-10-
20T10:00:00","estado":"PENDIENTE_PAGO","monto":150.00,"metodoPago":"TARJETA"}
```

```
{"id":6,"idCliente":1,"mascotaId":2,"mascotaNombre":"Michi","motivo":"Desparasitacion (pago-ms
apagado)","fechaHora":"2026-10-
21T09:00:00","estado":"PENDIENTE_PAGO","monto":60.00,"metodoPago":"EFECTIVO"} -> HTTP 201
{"id":7,"idCliente":1,"mascotaId":2,"mascotaNombre":"Michi","motivo":"Bano y corte (pago-ms
apagado)","fechaHora":"2026-10-
21T09:00:00","estado":"PENDIENTE_PAGO","monto":45.00,"metodoPago":"YAPE"} -> HTTP 201
```

Con el servicio de pagos apagado, `citas-ms` sigue respondiendo 201. Con Feign esta operación habría fallado o habría dependido de un *fallback*.

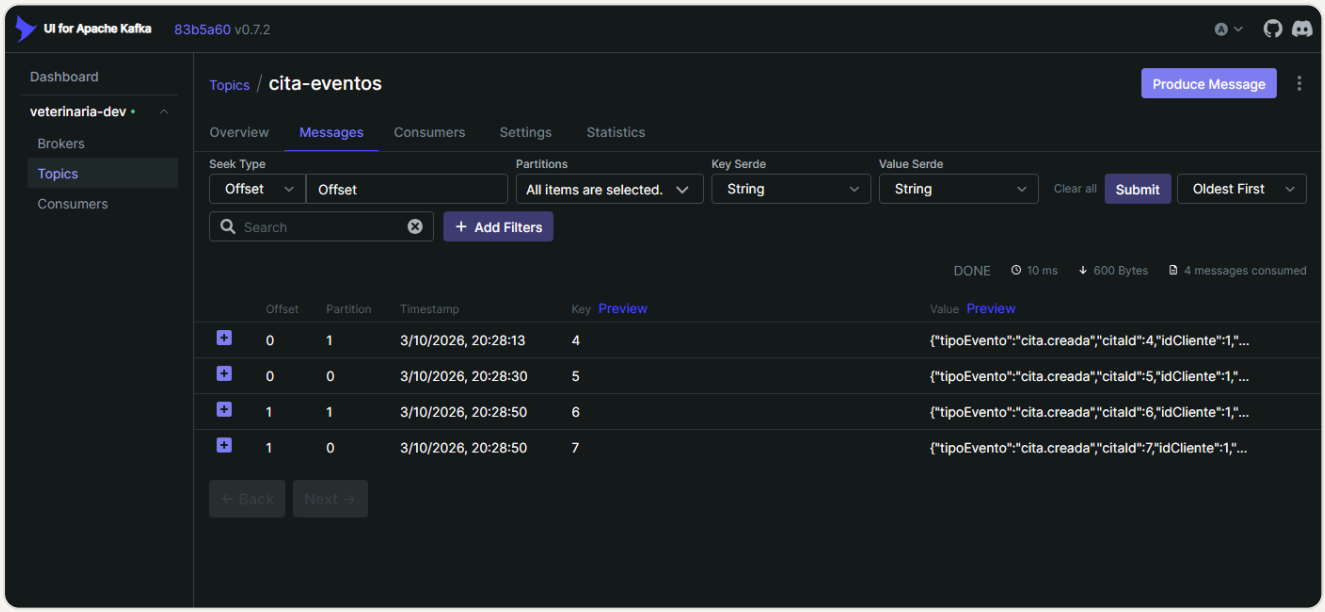
3. Los eventos esperan en el broker

```
$ kafka-consumer-groups.sh --describe --group veterinaria-pago-ms   (pago-ms apagado)
Consumer group 'veterinaria-pago-ms' has no active members.
GROUP               TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
CONSUMER-ID     HOST            CLIENT-ID
veterinaria-pago-ms cita-eventos    1          1               2               1
-               -               -
 id |     estado     | monto  | metodo_pago
----+----------------+--------+-------------
  5 | PENDIENTE_PAGO | 150.00 | TARJETA
  6 | PENDIENTE_PAGO |  60.00 | EFECTIVO
  7 | PENDIENTE_PAGO |  45.00 | YAPE
(3 rows)
```



Kafka UI: grupo `veterinaria-pago-ms` en estado **EMPTY** (0 miembros) con **lag 1** en la partición 1.

Kafka UI y `kafka-consumer-groups.sh` solo calculan el lag en particiones donde el grupo ya confirmó un offset (la partición 1). Los mensajes de las citas 5 y 7 están en la partición 0, donde el grupo aún no tenía offset. Ninguno se pierde: con `auto-offset-reset: earliest` el consumidor empieza desde el inicio de esa partición. La captura de mensajes lo confirma:



Tópico `cita-eventos` → Messages: los eventos de las citas 5, 6 y 7 publicados mientras pagos estaba apagado.

4. Se reinicia veterinaria-pago-ms

```
### 4. Se reinicia veterinaria-pago-ms y procesa los eventos acumulados
2026-10-03 20:28:14.286 [] INFO  p.e.u.pago.kafka.PagoEventosProducer - component=pago-producer
eventType=pago.validado citaId=4 status=PUBLICADO topic=pago-eventos partition=1 offset=0
2026-10-03 20:31:03.903 [] INFO  p.e.u.p.VeterinariaPagoApplication - Started
VeterinariaPagoApplication in 14.002 seconds (process running for 15.102)
2026-10-03 20:31:04.012 [] INFO  o.s.k.l.KafkaMessageListenerContainer - veterinaria-pago-ms:
partitions assigned: [cita-eventos-0, cita-eventos-1, cita-eventos-2]
2026-10-03 20:31:04.070 [] INFO  p.e.u.pago.kafka.CitaEventosConsumer - component=cita-consumer
eventType=cita.creada citaId=6 status=RECIBIDO partition=1 offset=1
```

```
2026-10-03 20:31:04.882 [] INFO  p.e.u.pago.service.PagoServiceImpl - component=pago-service
eventType=cita.creada citaId=6 status=PAGO_REGISTRADO pagoId=2 monto=60.00 metodoPago=EFECTIVO
2026-10-03 20:31:04.974 [] INFO  p.e.u.pago.kafka.CitaEventosConsumer - component=cita-consumer
eventType=cita.creada citaId=5 status=RECIBIDO partition=0 offset=0
2026-10-03 20:31:04.982 [] INFO  p.e.u.pago.service.PagoServiceImpl - component=pago-service
eventType=cita.creada citaId=5 status=PAGO_REGISTRADO pagoId=3 monto=150.00 metodoPago=TARJETA
2026-10-03 20:31:04.986 [] INFO  p.e.u.pago.kafka.CitaEventosConsumer - component=cita-consumer
eventType=cita.creada citaId=7 status=RECIBIDO partition=0 offset=1
2026-10-03 20:31:04.992 [] INFO  p.e.u.pago.service.PagoServiceImpl - component=pago-service
eventType=cita.creada citaId=7 status=PAGO_REGISTRADO pagoId=4 monto=45.00 metodoPago=YAPE
2026-10-03 20:31:05.000 [] INFO  p.e.u.pago.kafka.PagoEventosProducer - component=pago-producer
eventType=pago.validado citaId=6 status=PUBLICADO topic=pago-eventos partition=1 offset=1
2026-10-03 20:31:05.005 [] INFO  p.e.u.pago.kafka.PagoEventosProducer - component=pago-producer
eventType=pago.validado citaId=5 status=PUBLICADO topic=pago-eventos partition=0 offset=0
2026-10-03 20:31:05.005 [] INFO  p.e.u.pago.kafka.PagoEventosProducer - component=pago-producer
eventType=pago.validado citaId=7 status=PUBLICADO topic=pago-eventos partition=0 offset=1

### citas-ms recibe pago.validado y marca las citas
2026-10-03 20:31:05.002 [] INFO  p.e.u.c.kafka.PagoEventosConsumer - component=pago-consumer
eventType=pago.validado citaId=6 status=RECIBIDO partition=1 offset=1
2026-10-03 20:31:05.010 [] INFO  p.edu.upeu.citas.service.CitaService - component=cita-service
eventType=pago.validado citaId=6 status=CITA_PAGADA pagoId=2 estado=PENDIENTE_PAGO->PAGADA
2026-10-03 20:31:05.021 [] INFO  p.e.u.c.kafka.PagoEventosConsumer - component=pago-consumer
eventType=pago.validado citaId=5 status=RECIBIDO partition=0 offset=0
2026-10-03 20:31:05.023 [] INFO  p.edu.upeu.citas.service.CitaService - component=cita-service
eventType=pago.validado citaId=5 status=CITA_PAGADA pagoId=3 estado=PENDIENTE_PAGO->PAGADA
2026-10-03 20:31:05.027 [] INFO  p.e.u.c.kafka.PagoEventosConsumer - component=pago-consumer
eventType=pago.validado citaId=7 status=RECIBIDO partition=0 offset=1
2026-10-03 20:31:05.030 [] INFO  p.edu.upeu.citas.service.CitaService - component=cita-service
eventType=pago.validado citaId=7 status=CITA_PAGADA pagoId=4 estado=PENDIENTE_PAGO->PAGADA

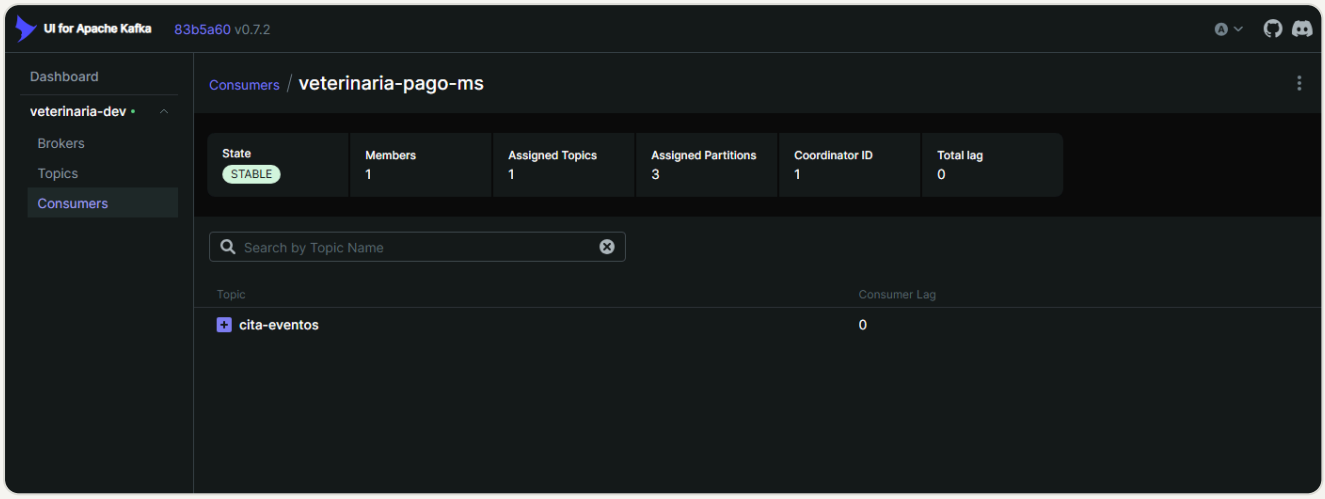
$ kafka-consumer-groups.sh --describe --group veterinaria-pago-ms   (despues del reinicio)
GROUP               TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
CONSUMER-ID
veterinaria-pago-ms cita-eventos    1          2               2               0
consumer-veteri
veterinaria-pago-ms cita-eventos    0          2               2               0
consumer-veteri
veterinaria-pago-ms cita-eventos    2          -               0               -
consumer-veteri

 id | estado | monto  | metodo_pago
----+--------+--------+-------------
  4 | PAGADA |  95.50 | YAPE
  5 | PAGADA | 150.00 | TARJETA
  6 | PAGADA |  60.00 | EFECTIVO
  7 | PAGADA |  45.00 | YAPE
(4 rows)

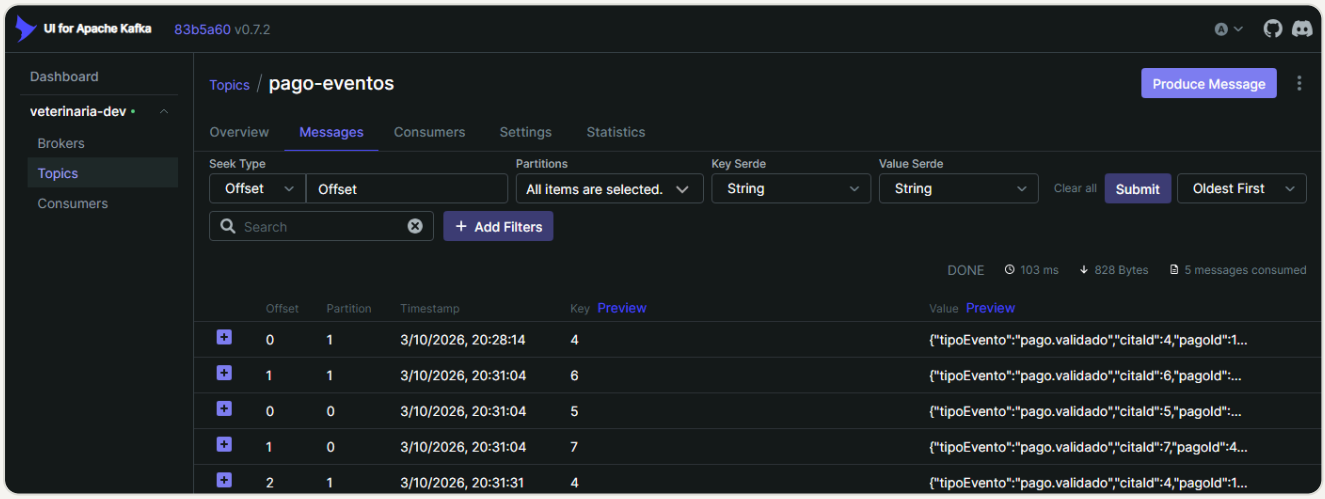
 id | cita_id | monto  | metodo_pago |  estado
----+---------+--------+-------------+----------
  1 |       4 |  95.50 | YAPE        | VALIDADO
  2 |       6 |  60.00 | EFECTIVO    | VALIDADO
  3 |       5 | 150.00 | TARJETA     | VALIDADO
  4 |       7 |  45.00 | YAPE        | VALIDADO
(4 rows)

$ curl http://localhost:18080/api/v1/pagos
[{"id":1,"citaId":4,"idCliente":1,"monto":95.50,"metodoPago":"YAPE","estado":"VALIDADO","fechaP
ago":"2026-10-03T20:28:14.050735"},
{"id":2,"citaId":6,"idCliente":1,"monto":60.00,"metodoPago":"EFECTIVO","estado":"VALIDADO","fec
```

haPago":"2026-10-03T20:31:04.824432"},
{"id":3,"citaId":5,"idCliente":1,"monto":150.00,"metodoPago":"TARJETA","estado":"VALIDADO","fechaPago":"2026-10-03T20:31:04.979439"},
{"id":4,"citaId":7,"idCliente":1,"monto":45.00,"metodoPago":"YAPE","estado":"VALIDADO","fechaPago":"2026-10-03T20:31:04.989937"}]



Después del reinicio: grupo **STABLE**, 1 miembro, 3 particiones asignadas y **lag total 0**.



Tópico `pago-eventos`: un `pago.validado` por cada cita. El último (cita 4, offset 2) viene de la prueba de duplicado.

Al volver, el servicio procesó los 3 eventos acumulados en ~1 segundo, el lag volvió a 0 y las tres citas pasaron a `PAGADA`.

10

Robustez del consumidor

Tres mensajes problemáticos se publicaron a mano en `cita-eventos` con `kafka-console-producer.sh`:

```
### a) Mensaje con JSON mal formado (key=99)
$ echo "99|esto-no-es-json" | kafka-console-producer.sh --topic cita-eventos --property
parse.key=true
### b) Evento duplicado: se reenvia cita.creada de la cita 4 (ya pagada)
### c) Tipo de evento no soportado en el topico

### Log veterinaria-pago-ms
2026-10-03 20:31:05.005 [] INFO  p.e.u.pago.kafka.PagoEventosProducer - component=pago-producer
eventType=pago.validado citaId=7 status=PUBLICADO topic=pago-eventos partition=0 offset=1
2026-10-03 20:31:28.956 [] ERROR o.s.k.listener.DefaultErrorHandler - Backoff
FixedBackOffExecution[interval=0, currentAttempts=1, maxAttempts=0] exhausted for cita-eventos-
0@2
2026-10-03 20:31:31.007 [] INFO  p.e.u.pago.kafka.CitaEventosConsumer - component=cita-consumer
eventType=cita.creada citaId=4 status=RECIBIDO partition=1 offset=2
2026-10-03 20:31:31.013 [] WARN  p.e.u.pago.service.PagoServiceImpl - component=pago-service
eventType=cita.creada citaId=4 status=DUPLICADO pagoId=1
2026-10-03 20:31:31.026 [] INFO  p.e.u.pago.kafka.PagoEventosProducer - component=pago-producer
eventType=pago.validado citaId=4 status=PUBLICADO topic=pago-eventos partition=1 offset=2
2026-10-03 20:31:33.107 [] INFO  p.e.u.pago.kafka.CitaEventosConsumer - component=cita-consumer
eventType=cita.cancelada citaId=8 status=RECIBIDO partition=0 offset=3
2026-10-03 20:31:33.108 [] INFO  p.e.u.pago.kafka.CitaEventosConsumer - component=cita-consumer
eventType=cita.cancelada citaId=8 status=IGNORADO motivo=tipo_no_soportado

### Log citas-ms
2026-10-03 20:31:05.030 [] INFO  p.edu.upeu.citas.service.CitaService - component=cita-service
eventType=pago.validado citaId=7 status=CITA_PAGADA pagoId=4 estado=PENDIENTE_PAGO->PAGADA
2026-10-03 20:31:31.026 [] INFO  p.e.u.c.kafka.PagoEventosConsumer - component=pago-consumer
eventType=pago.validado citaId=4 status=RECIBIDO partition=1 offset=2
2026-10-03 20:31:31.038 [] INFO  p.edu.upeu.citas.service.CitaService - component=cita-service
eventType=pago.validado citaId=4 status=SIN_CAMBIO estadoActual=PAGADA

 pagos_cita_4
--------------
            1
(1 row)

pago-ms health tras mensajes invalidos: HTTP 200
```

Causa registrada del mensaje inválido:

```
2026-10-03 20:31:28.956 ERROR o.s.k.listener.DefaultErrorHandler - Backoff
FixedBackOffExecution[interval=0, currentAttempts=1, maxAttempts=0] exhausted for cita-eventos-
0@2
Caused by: org.springframework.kafka.support.serializer.DeserializationException: failed to
deserialize
Caused by: org.apache.kafka.common.errors.SerializationException: Can't deserialize data  from
topic [cita-eventos]
Caused by: tools.jackson.core.exc.StreamReadException: Unrecognized token 'esto': was expecting
(JSON String, Number, Array, Object or token 'null', 'true' or 'false')
```

- **JSON mal formado:** `ErrorHandlingDeserializer` convierte el error en `DeserializationException`; `DefaultErrorHandler` lo registra y avanza al siguiente offset. El consumidor sigue vivo (health 200) y procesa los mensajes siguientes.
- **Duplicado:** Kafka garantiza entrega *al menos una vez*, así que un evento puede llegar dos veces. `PagoServiceImpl` busca por `cita_id` (UNIQUE) y no crea un segundo pago (`status=DUPLICADO`); `citas-ms` recibe otra vez `pago.validado` y no cambia nada (`SIN_CAMBIO`). Ambos consumidores son **idempotentes**.
- **Tipo no soportado:** `cita.cancelada` se registra como `IGNORADO` y no se procesa.

Métricas de Prometheus (`GET :8087/actuator/prometheus`) después de las pruebas:

```
$ curl -s localhost:8087/actuator/prometheus | grep -E
"records_consumed_total|spring_kafka_listener_seconds_count"
kafka_consumer_fetch_manager_records_consumed_total{client_id="consumer-veterinaria-pago-ms-
1",kafka_version="4.1.2",topic="cita-eventos"} 6.0
spring_kafka_listener_seconds_count{exception="DeserializationException",result="failure"} 1
spring_kafka_listener_seconds_count{exception="none",result="success"} 5
```

11

Preguntas de defensa

¿Cuál es la diferencia entre comunicación síncrona y asíncrona?

En la síncrona (Feign) el que llama espera la respuesta y necesita que el otro servicio esté disponible en ese momento; el resultado es inmediato. En la asíncrona el productor deja un mensaje en el broker y continúa; el consumidor lo procesa cuando puede. Se gana independencia de disponibilidad (sección 9: citas siguió respondiendo 201 con pagos caído) a cambio de consistencia eventual: la cita está unos instantes en `PENDIENTE_PAGO`.

¿Por qué hay clases de evento separadas en cada servicio?

Porque una librería compartida volvería a acoplar los servicios: un cambio en la clase obligaría a recompilar y desplegar a todos a la vez. Lo que se comparte es el **contrato** (el JSON documentado en la sección 7), no el código. Cada servicio tiene su copia y puede ignorar campos que no usa. Por eso además se desactivan las cabeceras de tipo: el consumidor no necesita conocer el nombre de clase Java del productor.

¿Qué garantías da Kafka sobre la entrega?

Con `acks=all` el productor recibe confirmación cuando el broker escribió el mensaje en el log. El mensaje se guarda en disco y queda disponible según la retención, aunque nadie lo haya leído todavía. El orden se garantiza **solo dentro de una partición**; por eso la key es el `citaId`. La entrega al consumidor es **al menos una vez**: si falla después de procesar y antes de confirmar el offset, el mensaje se vuelve a entregar. Por eso los consumidores son idempotentes.

¿Cómo recupera el servicio los mensajes si se cae?

El progreso de cada consumer group se guarda en el broker (tópico interno `__consumer_offsets`). Al reiniciar, el consumidor pide su último offset confirmado y continúa desde ahí; si en una partición no tiene offset, usa `auto-offset-reset: earliest`. Es lo que muestra la sección 9: los eventos de las citas 5, 6 y 7 se procesaron al volver.

¿Qué es el offset del consumer group y por qué importa?

Es la posición del siguiente mensaje que el grupo debe leer en cada partición. La diferencia entre el último offset del tópico y el offset confirmado es el **lag**: cuántos mensajes faltan procesar. Importa porque permite reanudar sin perder ni repetir trabajo, y porque cada grupo tiene sus offsets independientes: otro servicio que se suscriba a `cita-eventos` con su propio grupo recibirá todos los mensajes sin afectar a `veterinaria-pago-ms`.

12

Criterios de aceptación

Criterio (4.4)	Evidencia	
Ambos tópicos con 3 particiones	Sección 4, `--describe` y Kafka UI	cumple
pago-ms consume `cita.creada` y publica `pago.validado`	Sección 8, logs y mensajes	cumple
citas-ms pasa la cita a `PAGADA` al recibir el pago	Sección 8, REST y BD	cumple
Desacoplamiento: registro con pagos apagado y proceso al reiniciar	Sección 9, lag 1 → 0	cumple
Logs con `component`, `eventType`, `citaId`, `status`	Secciones 8–10	cumple
Sin librerías de eventos compartidas	Sección 7, paquetes `event` separados	cumple
Endpoints REST funcionales	`GET /api/v1/pagos`, `/{id}` (200 y 404)	cumple

Decisiones de diseño

- **Publicar después del commit** (`TransactionSynchronization.afterCommit`): si el INSERT hace rollback no sale un evento "fantasma". Límite conocido: si el proceso muere entre el commit y el envío, el evento se pierde; el patrón *Transactional Outbox* lo resolvería (fuera del alcance de S8).
- **Key = citaId**: todos los eventos de una cita van a la misma partición y se procesan en orden.
- **Idempotencia**: `cita_id UNIQUE` en pagos y transición solo desde `PENDIENTE_PAGO` en citas.
- **Ambos servicios declaran los tópicos** (`KafkaTopicsConfig`): citas-ms puede publicar aunque pagos nunca haya arrancado. Si el tópico ya existe, Spring no cambia sus particiones.
- **Tópico de pruebas separado** para el script Python, para no mezclar datos de prueba con pagos reales.
- Las citas que quedan en `PENDIENTE_VALIDACION` (Circuit Breaker abierto) no publican evento: no se cobra una cita cuya mascota no se pudo validar.

Durante la práctica los puertos 19092, 15432 y 15435 estaban ocupados por contenedores de otro proyecto. Se detuvieron con `docker stop` durante las pruebas y se volvieron a iniciar al terminar.

Conclusiones

- Kafka desacopla la disponibilidad: el registro de citas no depende de que pagos esté en línea, y los eventos pendientes se procesan al volver el servicio.
- El costo es la consistencia eventual y la necesidad de diseñar consumidores idempotentes, porque la entrega es al menos una vez.
- Comunicación síncrona y asíncrona conviven: Feign para lo que se necesita al instante (validar la mascota) y eventos para lo que puede ocurrir después (el pago).
- El contrato documentado reemplaza a la librería compartida como punto de acuerdo entre equipos.

Bibliografía

- Apache Software Foundation. (2025). *Apache Kafka Documentation*. https://kafka.apache.org/documentation/
- Spring. (2025). *Spring for Apache Kafka — Reference*. https://docs.spring.io/spring-kafka/reference/
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media. Cap. 11: Stream Processing.
- Richardson, C. (2018). *Microservices Patterns*. Manning. Cap. 3: Interprocess communication (Transactional Outbox).
- Provectus. (2024). *UI for Apache Kafka*. https://github.com/provectus/kafka-ui